Contamination Detection via Wheeler Graphs

Samer Aslan[*]
Johns Hopkins University
saslan1@jh.edu

Owen Bianchi[*]
Johns Hopkins University
obianch1@jh.edu

Anirudh Kashyap[*]
Johns Hopkins University
akashy11@jh.edu

Senapati Diwangkara[*]
Johns Hopkins University
ssangdi1@jhu.edu

[*]Authors Contributed Equally

December 2023

Abstract

Contamination in genomic sequencing often arises from the inadvertent inclusion of extraneous DNA, which can lead to false interpretations of genetic data. This underscores the importance of reliable detection methods in ensuring data integrity. Existing methods for this task can be slow at query time and may not identify the contaminant organisms at play. Our project attempts to combat this using wheeler graphs, presenting a new reference-guided approach for contamination detection in genomic datasets. Contrasting with traditional reference-based methods that typically rely on searching for specific genes across multiple, separate data structures derived from reference FASTA files, our method employs the creation of a Wheeler graph to enable efficient and accurate contamination detection. This approach allows for the integration of multiple FASTA files into a single Wheeler graph, enhancing space efficiency, while also identifying the contaminant organism. By efficiently querying this unified graph structure, we aim to enhance the ability to identify the sources of specific reads, enabling more accurate identification of potential contaminants. We also explore prior reference-free and reference-guided methods to broaden our understanding of contamination detection and lay the foundation of what we need to overcome.

1 Introduction

The field of genomics has witnessed exponential growth, largely due to declining sequencing costs and advancements in sequencing technology. Naturally, such growth has brought new challenges, including contamination in genomic sequences. Such contamination, often undetected and pervasive, can compromise the integrity of genomic datasets, leading to inaccurate analyses and conclusions. An example of this issue is highlighted in the study "Mycoplasma contamination in the 1000 Genomes Project" [Langdon, 2014], where it was discovered that at least 7% of the samples in the 1000 Genomes Project were contaminated with Mycoplasma. This finding underscores the critical need for robust methods to detect and analyze contamination in genomic data, especially considering the scale and impact of projects like the 1000 Genomes Project on the field of genomics.

In response to this need, our project introduces a novel approach utilizing Wheeler graphs [Gagie et al., 2017] for contamination detection in genomic datasets. Wheeler graphs, efficient data structures for string processing, are not usually known to be used for this contamination detection use case. However, we sought to leverage the ability of wheeler graphs to effectively handle sequence repeats to improve the space complexity of detecting *and* identifying contaminants. By incorporating multiple host and contaminant genomes into a unified Wheeler graph, our method improves the space efficiency of the contamination detection task, while also allowing for organism identification in the case of multiple contaminants.

Our methodology is inspired by recent advancements in genomic analysis tools, the revelations of contamination in major genomic projects, and the existing 'gramtools' software [Letcher et al., 2021] which uses genome graphs for genetic variation analysis. Drawing from the lessons learned in studies like Langdon's, we seek to build upon the concept of genome graphs, as implemented in 'gramtools', but extend it further using Wheeler graphs for a more efficient and comprehensive analysis of genomic contamination. By efficiently managing common prefixes and suffixes, our Wheeler graph-based approach aims to accurately pinpoint the specific sources of contamination, a task that has become increasingly crucial in the wake of findings like those of Langdon. With this approach, we set out to achieve two main goals: firstly, the construction of a Wheeler graph from a target genome's FASTA file to identify contaminated reads in a FastQ file, and secondly, the creation of a Wheeler graph representing multiple distinct organisms, to facilitate a more intricate analysis of contamination.

2 Prior Works

Genomic contamination detection has seen diverse methodologies being employed over the years, each with its unique strengths and weaknesses. These methodologies can be broadly categorized into reference-independent and reference-dependent approaches, each taking a different approach in analyzing genomic

sequences. Reference-independent tools do not rely on predefined databases, instead analyzing DNA sequences based on their intrinsic properties. On the other hand, reference-dependent tools require comprehensive databases against which sequences are classified and assessed. This section delves into various tools and strategies within these categories, highlighting their operational mechanisms and specific application scenarios. Additionally, a focus is given to the emerging utilization of Wheeler graphs in this domain, a novel approach that could promise advancements in the field.

Reference-independent tools are characterized by their reliance on the intrinsic properties of DNA sequences, without the need for a pre-existing database for analysis. For example, BlobTools [Kumar et al., 2013] , which utilizes GC content and read coverage to represent data in a two-dimensional plot, color-coded based on the NCBI taxonomy as provided by the user. Another tool, Anvi'o [Eren et al., 2015] , clusters contigs based on their k-mer frequencies and read coverage, and employs circo plots for visual representation. In addition, tools such as ProDeGe and PhylOligo [Mallet et al., 2017] use k-mer binning techniques. ProDeGe Tennessen et al., 2016, for instance, incorporates taxonomical information for calibration, whereas PhylOligo allows users to select k-mers they believe correspond to the target organism, represented through a neighbor-joining tree.

On the other hand, reference-dependent tools require a database for sequence classification and contamination assessment. This category includes gene marker-based estimators such as SINA [Pruesse et al., 2012], which assesses the number of SSU rRNA genes present in a genome. As these gene markers are typically present in a single copy, their multiple occurrences are indicative of contamination. Extending beyond this, some tools also utilize protein reference databases for a more comprehensive analysis. Further, several tools adopt phylogenetic placement strategies combined with specific gene markers to detect contamination.

Among the genome-wide approaches, the entire genome is aligned against a reference database. Tools like Kraken [Wood and Salzberg, 2014] classify genomic regions by dividing them into k-mers and mapping these onto a phylogenetic tree. In contrast, CONSULT Rachtman et al. [2021] employs the Hamming distance for k-mer comparison rather than exact matching. Moreover, tools such as BASTA [Kahlke and Ralph, 2018] classify sequences using lowest common ancestor labels, with sequences BLASTed against a taxonomically labeled database.

A key aspect in the analysis of genomic sequences is the distinction between positive and negative filtering methodologies, as utilized by several tools in genomic research. Negative filtering, employed by tools such as BlobTools, Anvi'o, ProDeGE, PhylOligo, Forty-Two, Conterminator, BASTA involves the exclusion of sequences whose taxonomy diverges from an expected taxon. This expected taxon is either specified by the user or automatically determined by the tool, identifying contaminants based on taxonomic discrepancies. On the other hand, positive filtering, available in Kraken, CLARK, BASTA and is characterized by retaining sequences that match a specific taxonomy. This approach

is particularly beneficial when the taxonomy of the sequenced organism is already known, or for further analysis post-taxonomy overview of sequences in the genome.

The Wheeler graph was first introduced by Gagie et al.[Gagie et al., 2017]. These graphs are characterized by their ability to efficiently handle string data, making them particularly suitable for tasks such as pattern matching and sequence alignment. In the context of contamination detection, we think that Wheeler graphs offer a unique advantage; they can represent multiple genomic sequences within a single, space-efficient data structure. This feature is particularly beneficial when dealing with large-scale genomic datasets, where conventional methods may require large storage requirements. Our methodology leverages this advantage, building upon the foundational work of [Chao et al., 2023] and others, to create a comprehensive system for detecting, identifying, and analyzing genomic contamination. By integrating the principles of Wheeler graphs with querying techniques, we aim to provide a more efficient means of contamination detection than previously possible, while also identifying the contaminant organisms themselves. This means that, in practice, if we have multiple suspected contaminant organisms, the wheeler graph can be built with these contaminant organisms and target FASTA files. Then, when querying a given read, the wheeler graph will return the organism associated with that read.

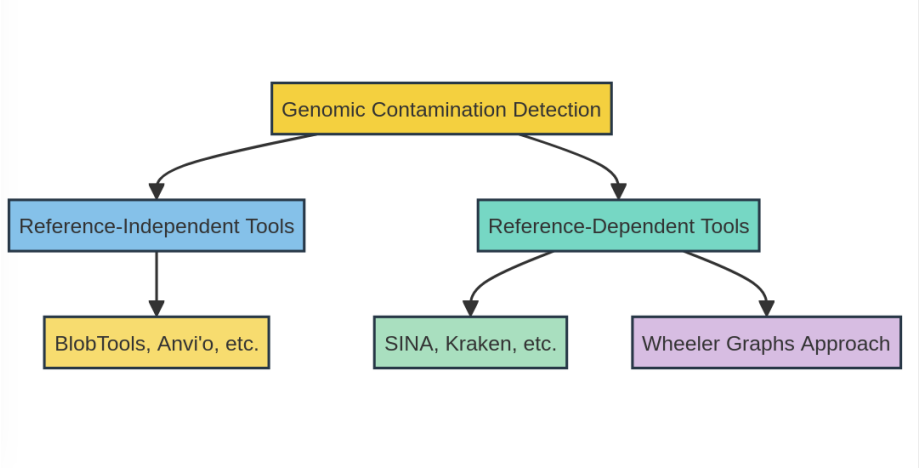


Figure 1: Flowchart Depicting Genomic Contamination Detection Methods

3 Methods and Software

3.1 Dataset Creation

3.1.1 Synthetic DNA Sequence Generation

We developed the dataset in two ways. First a synthetic DNA sequence dataset was constructed using a two-phase process in Python. In the first phase, the `generate_sequence` function synthesized sequences with specified lengths, GC contents, and particular motifs. Here, we picked specific parameters such that the reads should be easily identifiable. These sequences were then written into FASTA files for different sequence types (host, contaminant1, and contaminant2), each with unique lengths and characteristics we specified earlier, using the `generate_fasta` function. Furthermore, the `generate_fastq` function converted these FASTA files into a single comprehensive FASTQ file. This function processed each sequence to extract subsequences of a predetermined read length (50 bases) and assigned mock quality scores to them. The final FASTQ file, named `'mock_data.fastq'`, compiled 500 reads per sequence type, each clearly labeled to indicate its source. This allowed for us to cross reference any classifications in the future with the ground truth, enabling evaluation of the classifier.

3.1.2 Simulation of Human and Mycoplasma Reads

In this method, a simulation was conducted to generate single-end reads from both the human and Mycoplasma genomes, followed by a mixing step to model Mycoplasma contamination. Reads of 100 base pairs (bp) with 20-fold coverage were generated from the human genome using the ART Illumina tool with the HiSeq 2500 sequencing system. A similar procedure was executed for the Mycoplasma genome, producing reads designated as 'mycoplasma_reads'. Subsequently, a custom Python script, `mix_reads`, was utilized to intermix these reads, creating a composite dataset. This script was configured to introduce Mycoplasma reads into the human reads at a specified contamination rate, resulting in a realistic dataset ideal for testing and validating contamination detection methods.

3.2 Graph Generation

In a traditional alignment problem, the text sequence is represented as a FASTA file, in which the organism's genome is sequenced. In order to represent multiple organisms however, we need to devise a way to convert multiple FASTA file that each corresponds to different organism to a single data structure, in this case, a Wheeler Graph.

In particular, we choose to represent our text sequence in a DeBruijn graph, which is one of many kinds of Wheeler Graphs.

We follow the standard DeBruijn construction, in which given a sequence and a k parameter, we would form a DeBruijn graph in which each node represents $k-1$-mer substring of the genome and each edge represents a suffix-prefix relationship.

However, we modified the nodes to also store the organism ID to signify which FASTA file this node is coming from. This is stored as a set to accommodate multiple organism that might share the same $k-1$-mer.

First, we read multiple FASTA files and process each $k-1$-mer substring into a set. Next, we enrich this set with an organism ID by moving it into a dictionary, with the key as its key and the $k-1$-mer as the value. Finally, we construct the graph by constructing the node from the dictionary and adding the edge according to the suffix-prefix relationship. We store this graph in a DOT format, and we store the organism ID in the edge label.

Below is the pseudocode of the graph generation algorithm:

Algorithm 1 Wheeler Graph Generation Algorithm

Input
  F_x FASTA file for organism x
  k k parameter of DeBruijn graph
Output
  G Wheeler Graph
$kmer_d$ = empty dictionary ▷ Variable to associate $k-1$-mer with organism
for $F_a \forall a \in x$ **do** ▷ Iterate through organism
  $kmer_s$ = empty set ▷ Variable to store all possible $k-1$-mer
  for $i \in 0$ to length of $F_a - k + 1$ **do**
    add $F_a[i : i + k - 1]$ to $kmer_s$
  end for
  for $kmer \in kmer_s$ **do** ▷ Check if we've seen this kmer in other organism
    if $kmer \in kmer_d$ **then**
      add new organismID to node
    else
      new node
    end if
  end for
  for $i \in 0$ to length of $F_a - k + 1$ **do** ▷ Make some edges on adjacency
    $kmer_d[F_a[i : i + k - 1]].nextnode = kmer_d[F_a[i + 1 : i + k]]$
  end for
end for
Return $kmer_d$ in DOT format where the organism ID is written as label

3.3 Querying The Graph

After we generate a single Wheeler Graph from multiple FASTA files that represents different organisms, we want to use the resulting graph to check which

organism could match a given read (represented by a FASTQ file) the best. In other words, we frame the graph querying process as an alignment process in order to gain knowledge of which organism a given sequence read might come from.

In order to do that, we borrow the approach of FM index and extend them to handle branching in the graph. Given a FASTQ file as the input, we will iterate through the reads and align them against the graph.

Just like FM index, from a single read, we will align progressively longer suffixes. However, unlike FM index, one substring from a single 'row' in the Burrows-Wheeler Matrix could be extended into multiple longer substrings, due to a branch that might exist in the relevant path of the graph.

More concretely, we start by aligning the last character of the pattern. We search for the edges in the wheeler graph whose label is the same as the last character of the read, and store it in a buffer variable. Next, from this buffer variable, we start aligning the second-to-last character of the pattern by looking at the outgoing edge of the node pointed by the edges in this buffer variable. We update the buffer variable by filtering the path that adheres to this second alignment. Importantly, if two edges in the buffer have the same second edge in the alignment (those two edges joined), then we count those as two paths, not one. We repeat this process until all the character in the pattern has been aligned. If at any point, no appropriate edge is found, then we stop the alignment process since no further alignment could be found. This is a signal that the pattern might come from other organisms other than those represented by the graph. Additionally, since we want to identify which organism(s) correspond(s) to the path, we are also concerned with checking the organism(s) for each edge. With the addition of each edge, we take the intersection of the organismIDs for the current edge and the additional edge. For example, if the current edge has an organismID set of {1, 2, 4} and the additional edge has an organismID set of {1, 3, 4} then we extend the path, but it now has organismIDs {1, 4}. The result of this query algorithm are the paths in the graph that represent possible alignments along with the organism(s) corresponding to these paths.

Below is the pseudocode of the query algorithm:

Algorithm 2 Wheeler Graph Organism ID Query Algorithm

Input
G Wheeler graph
p A string of pattern to align
Output
ids Organism ids that have this pattern

paths = empty array ▷ Buffer variable to store array of string of nodes
for $edge \in G.edges$ **do** ▷ Get edges that match with first char of p
  if $edge.label = p[0]$ **then**
    append $([edge.src, edge.dst], edge.organismID)$ to $paths$
  end if
end for
for $char \in p[1]$ **do** ▷ Continue path if pattern and organismID matches
  for $(path, orgID) \in paths$ **do**
    for $edge \in G.edges$ if $edge.src = path[-1]$ **do**
      if $edge.label = char$ **then**
        Append $edge$ to $path$ if $edge.organismID \in orgID$
        Update $orgID$ with its intersection
      end if
    end for
  end for
end for
Return $paths[0][1]$ ▷ Return the organismID if found

3.4 Alignment using Bowtie2 to check for Contaminants

In this study, we aimed to evaluate the efficiency of a fast aligner, Bowtie2, in distinguishing between human and Mycoplasma reads. To achieve this, we simulated single-end reads from the human genome using HiSeq 2500 system parameters and generated reads from the Mycoplasma genome. These reads were mixed to simulate a 5% contamination rate of Mycoplasma, using a custom Python script. Subsequently, the reads were aligned to Mycoplasma and human genome indices constructed with Bowtie2.

4 Results

Our research project aimed to develop and test a novel genomic contamination detection method using Wheeler Graphs. To evaluate the efficacy of our method, we performed two sets of experiments: one using a mock dataset and the other with real dataset samples. We then did an experiment of Mycoplasma detection using Bowtie2 for comparison purposes.

4.1 Experiment 1 : Mock Dataset Results

The testing process was enabled by the test.py script which automates the creation of synthetic DNA sequences, the construction of Wheeler Graphs from these sequences, and the subsequent querying of these graphs to identify potential contaminants. The process is as follows:

Generation of Synthetic DNA Sequences Using the generate_sequence and generate_fasta functions, we created synthetic DNA sequences with predetermined GC contents and motifs. This approach allowed us to have full control over the composition of the datasets, thus ensuring the reliability of our test results.

Construction of Wheeler Graphs The build_wheeler_graph function was used to construct Wheeler Graphs from the generated FASTA files. These graphs included nodes for k-1-mers and edges representing suffix-prefix relationships, with each node additionally storing an organism ID.

Querying the Graphs The query_reads function was employed to align reads from the mock FASTQ file against the Wheeler Graph. This process was used to classify the source organism of each read, distinguishing between host and both contaminant sequences.

Testing Parameters For our experiment, we intentionally selected values for GC contents and specific motifs to ensure that our Wheeler Graph-based method could effectively handle distinct genomic sequences. We simulated a scenario with one host and two contaminants, mirroring realistic conditions where a genomic dataset might be contaminated with multiple foreign DNA sources.

Category	Total Ground Truth	Total Predicted	Precision	Recall	F1 Score	Accuracy
Host	500	518	0.97	1.00	0.98	0.99
Contaminant1	500	229	1.00	1.00	1.00	1.00
Contaminant2	500	218	1.00	0.92	0.96	0.99
Ambiguous	0	535	-	-	-	-

Figure 2: Results of Experiment 1

Results and Analysis Upon executing the test.py script, the following results were observed. A total of 1500 reads were recognized, indicating the Wheeler Graph's capacity to handle and analyze a substantial number of genomic sequences. High precision scores (0.97 for the host, 1.00 for both contaminants) demonstrate the method's accuracy in correctly identifying true positive

reads. The recall score was perfect (1.00) for the host and the first contaminant, indicating no false negatives. For the second contaminant, the recall was slightly lower (0.92), suggesting some slight room for improvement. The F1 scores and accuracy rates were consistently high, underscoring the overall effectiveness of our approach in contamination detection. A number of reads (535) were classified as ambiguous, highlighting an area for potential improvement. Reducing ambiguity in read classification will be a focus in future iterations of our method.

4.2 Experiment 2: Real Dataset Results

In Experiment 2, we ventured into the practical application of our Wheeler Graph-based method on real genomic data. This step was crucial for understanding the scalability and adaptability of our approach in real-world scenarios. We utilized publicly available genomic sequences, selecting a kangaroo gene as the host National Center for Biotechnology Information (NCBI) [1999] and two different Mycoplasma species as contaminants National Center for Biotechnology Information (NCBI) [2011b,a]. The primary objective was to test the robustness of our method in a more complex and diverse genomic environment compared to the controlled conditions of the mock dataset.

Methodology The methodology mirrored that of Experiment 1, involving the generation of a Wheeler Graph from the FASTA files of the selected genomes and subsequent querying of this graph with real sequencing reads. This process was expected to highlight the efficiency of our method in a real-world setting, providing insights into its practical applicability and limitations.

Challenges Encountered During the course of Experiment 2, several challenges were encountered, primarily revolving around the complexity and variability inherent in real genomic datasets. These challenges included the handling of highly diverse genetic sequences and the accurate differentiation between closely related species, which are common in real genomic samples. These factors introduced a level of complexity not encountered in the controlled environment of the mock dataset.

Results and Interpretation The results of Experiment 2 presented a mixed outcome. While the Wheeler Graph successfully recognized all of the reads (1500), the differentiation between the host and contaminants was less clear-cut compared to the mock dataset. The majority of classifications were ambiguous, and the identification of contaminants was not as accurate as anticipated. These results underscore the complexities involved in real-world genomic data analysis and highlight areas for further refinement in our methodology.

4.3 Experiment 3: Alignment Statistics using Bowtie2

The alignment statistics revealed that out of 911,490 processed reads, only 0.96% aligned to the Mycoplasma genome, including 8,475 (0.93%) reads aligning exactly once and 245 (0.03%) reads aligning more than once. This indicates that the majority of the reads were from the human genome.

5 Conclusions

The results from Experiment 1 on the mock dataset demonstrate the viability of using Wheeler Graphs for genomic contamination detection. The high precision, recall, and accuracy rates from this experiment indicate that our method can effectively differentiate between host and contaminant DNA, even in complex genomic datasets. Experiment 2, while presenting mixed results, provided valuable insights into the application of Wheeler Graphs in real-world genomic scenarios. These findings underscore the method's potential as a tool for genomic contamination detection.

The practical implementation of our Wheeler Graph-based method in Experiment 2, although less successful than anticipated, was instrumental in understanding the adaptability of our approach in diverse genomic environments. This step is crucial in our journey towards developing a robust tool for genomic contamination detection and lays a foundation for future improvements.

In addressing the ambiguity observed in our experiments, particularly the classification of reads from multiple organisms, we recognize the need for further refinement of our methodology. Future work will focus on minimizing the number of ambiguous classifications by enhancing the algorithm's ability to differentiate between closely related species. Additionally, the integration of approximate matching techniques should be used to improve the identification of contaminants in diverse genomic datasets. These efforts aim to improve the scalability and precision of contamination detection, ensuring our method's applicability to a broader range of genomic datasets. By tackling these challenges, we aim to enhance the accuracy and reliability of our Wheeler Graph-based approach. In essence, we find that the wheeler graph approach for contamination detection and identification is promising, and with further refinement, could prove to be a robust tool for this task, even with real-world datasets.

6 Code Availability

The code for this project can be found in our GitHub repository, accessible here.

7 Author Contributions

Owen Contributed to the writing of the abstract, introduction, prior works, conclusion, and overall formatting of the paper. Conducted and detailed ex-

periments 1 and 2. Wrote the code for the generate_mock_genomic_data.py and test.py.

Samer Wrote the code for build and query. Managed the repository and wrote the ReadMEs. Wrote parts of abstract, intro, related works, building and querying methods, and conclusion.

Diwang Written the code for querying the graph. Written the graph generation and graph querying (3.2 and 3.3) section of the paper.

Anirudh Contributed to the writing of the abstract, introduction, prior works. Created a simulated experiment to test mycoplasma contamination in human genome.

All Members Contributed to problem solving and editing, revising, and formatting of the paper.

References

Kuan-Hao Chao, Pei-Wei Chen, Sanjit A. Seshia, and Ben Langmead. Wgt: Tools and algorithms for recognizing, visualizing and generating wheeler graphs. *bioRxiv*, 2023. doi: 10.1101/2022.10.15.512390. URL `https://www.biorxiv.org/content/early/2023/04/12/2022.10.15.512390`.

A. Eren, Özcan Esen, Christopher Quince, Joseph Vineis, Hilary Morrison, Mitchell Sogin, and Tom Delmont. Anvi'o: An advanced analysis and visualization platformfor 'omics data. *PeerJ*, 3:e1319, 10 2015. doi: 10.7717/peerj.1319.

Travis Gagie, Giovanni Manzini, and Jouni Sirén. Wheeler graphs: A framework for bwt-based data structures. *Theoretical Computer Science*, 698: 67–78, 2017. ISSN 0304-3975. doi: https://doi.org/10.1016/j.tcs.2017.06.016. URL `https://www.sciencedirect.com/science/article/pii/S0304397517305285`. Algorithms, Strings and Theoretical Approaches in the Big Data Era (In Honor of the 60th Birthday of Professor Raffaele Giancarlo).

Tim Kahlke and Peter J. Ralph. Basta – taxonomic classification of sequences and sequence bins using last common ancestor estimations. *Methods in Ecology and Evolution*, 10(1):100–103, 2018. doi: 10.1111/2041-210X.13095. URL `https://doi.org/10.1111/2041-210X.13095`.

Sujai Kumar, Martin Jones, Georgios Koutsovoulos, Michael Clarke, and Mark Blaxter. Blobology: exploring raw genome data for contaminants, symbionts and parasites using taxon-annotated gc-coverage plots. *Frontiers in Genetics*, 4, 2013. ISSN 1664-8021. doi: 10.3389/fgene.2013.00237. URL `https://www.frontiersin.org/articles/10.3389/fgene.2013.00237`.

William B Langdon. Mycoplasma contamination in the 1000 genomes project. *BioData Mining*, 7(3), 2014. doi: 10.1186/1756-0381-7-3. URL https://biodatamining.biomedcentral.com/articles/10.1186/1756-0381-7-3.

B. Letcher, M. Hunt, and Z. Iqbal. Gramtools enables multiscale variation analysis with genome graphs. *Genome Biology*, 22(259), 2021. doi: 10.1186/s13059-021-02474-0. URL https://doi.org/10.1186/s13059-021-02474-0.

Ludovic Mallet, Tristan Bitard-Feildel, Franck Cerutti, and Hélène Chiapello. PhylOligo: a package to identify contaminant or untargeted organism sequences in genome assemblies. *Bioinformatics*, 33(20):3283–3285, 06 2017. ISSN 1367-4803. doi: 10.1093/bioinformatics/btx396. URL https://doi.org/10.1093/bioinformatics/btx396.

National Center for Biotechnology Information (NCBI). Macropus sp. MeCP2 gene 3'UTR, GenBank: AJ132919.1. https://www.ncbi.nlm.nih.gov/nuccore/AJ132919.1, 1999. [Online].

National Center for Biotechnology Information (NCBI). Mycoplasma leachii 99/014/6 complete genome, GenBank: FR668087.1. https://www.ncbi.nlm.nih.gov/nuccore/FR668087.1, 2011a. [Online].

National Center for Biotechnology Information (NCBI). Synthetic Mycoplasma mycoides JCVI-syn1.0 clone sMmYCp235-1, complete sequence, GenBank: CP002027.1. https://www.ncbi.nlm.nih.gov/nuccore/CP002027.1, 2011b. [Online].

Elmar Pruesse, Jörg Peplies, and Frank Oliver Glöckner. SINA: Accurate high-throughput multiple sequence alignment of ribosomal RNA genes. *Bioinformatics*, 28(14):1823–1829, 05 2012. ISSN 1367-4803. doi: 10.1093/bioinformatics/bts252. URL https://doi.org/10.1093/bioinformatics/bts252.

Eleonora Rachtman, Vineet Bafna, and Siavash Mirarab. CONSULT: accurate contamination removal using locality-sensitive hashing. *NAR Genomics and Bioinformatics*, 3(3):lqab071, 08 2021. ISSN 2631-9268. doi: 10.1093/nargab/lqab071. URL https://doi.org/10.1093/nargab/lqab071.

Kristin Tennessen, Evan Andersen, Scott Clingenpeel, Christian Rinke, Derek S. Lundberg, James Han, Jeff L. Dangl, Natalia Ivanova, Tanja Woyke, Nikos Kyrpides, and Amrita Pati. Prodege: a computational protocol for fully automated decontamination of genomes. *ISME J*, 10(1):269–272, 2016. doi: 10.1038/ismej.2015.100.

Derrick E. Wood and Steven L. Salzberg. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*, 15:R46, 2014. doi: 10.1186/gb-2014-15-3-r46.